# CLEARS

Climate-Linked Evaluation and Analytics for Resilience in Airport Systems

Complete Team Task Assignment & Project Execution Guide

**Person 1**
Data & Analytics Lead

**Person 2**
ML & Explainability Lead

**Person 3**
App & Dashboard Lead

**Person 4**
Documentation & Support

**Milestone 1**
Foundation

**Milestone 2**
Core Build

**Milestone 3**
Integration

## Table of Contents

1. Project Overview & Pipeline
2. Person 1 — Data & Analytics Lead
Milestone 1 Tasks (1.1 – 1.4)
Milestone 2 Tasks (1.5 – 1.7)
Milestone 3 Tasks (1.8)
3. Person 2 — ML & Explainability Lead
Milestone 1 Tasks (2.1 – 2.2)
Milestone 2 Tasks (2.3 – 2.7)
Milestone 3 Tasks (2.8)
4. Person 3 — App & Dashboard Lead
Milestone 1 Tasks (3.1 – 3.2)
Milestone 2 Tasks (3.3 – 3.7)
Milestone 3 Tasks (3.8)
5. Person 4 — Documentation & Support Lead
Milestone 1 Tasks (4.1 – 4.3)
Milestone 2 Tasks (4.4 – 4.6)
Milestone 3 Tasks (4.7 – 4.8)
6. Collaboration Rules & Shared Conventions
7. Codebase Structure Reference

## 1. Project Overview & Pipeline

CLEARS is an end-to-end decision-support system that uses machine learning, explainable AI, and data analytics to predict weather-driven airport disruptions, score airport resilience, simulate future scenarios, and present everything in an interactive operational dashboard. The pipeline has ten layers, each owned by a specific team member.

| # | Layer | Owner | Output |
|---|---|---|---|
| 1 | Data Collection | Person 1 | Raw CSV datasets |
| 2 | Data Cleaning | Person 1 | Cleaned merged dataframe |
| 3 | Feature Engineering | Person 1 | Feature matrix + dictionary |
| 4 | Disruption Prediction Model | Person 2 | disruption_model.pkl |
| 5 | Delay Severity Estimation | Person 2 | severity_model.pkl |
| 6 | Explainability (SHAP) | Person 2 | explain.py + SHAP plots |
| 7 | Resilience Index | Person 1 + 2 | resilience_scores.csv |
| 8 | Scenario Simulation | Person 2 + 3 | scenario_risk() function |
| 9 | Recommendation Engine | Person 3 | recommend.py |
| 10 | Dashboard & Visualisation | Person 3 | streamlit_app.py |

## 2. Person 1 — Data & Analytics Lead

Person 1 is the foundation of the entire project. Every other team member's work depends on the quality, structure, and documentation of the data pipeline. This role requires careful attention to data integrity, clear column naming conventions, and thorough documentation so that Persons 2 and 3 can consume the outputs without ambiguity. Person 1 also defines the business KPIs that give the project its analytics dimension.

**Milestone 1 — Foundation**

### T-1.1 Identify and Source Datasets

**Goal**

Find and download the raw weather and flight data that powers the entire project. You need at minimum: (1) historical hourly weather readings covering temperature, wind speed, visibility, humidity, rainfall, and cloud cover; and (2) flight departure/arrival delay records linked to the same airports.

**Recommended Sources**

- NOAA Integrated Surface Database (ISD) — hourly weather from airport weather stations worldwide. URL: https://www.ncdc.noaa.gov/isd. Download as CSV. Filter to your chosen airports by their ICAO station code.
- BTS On-Time Performance (US airports only) — monthly flight-level delay data. URL: https://www.transtats.bts.gov. Select fields: FlightDate, Origin, Dest, DepDelay, ArrDelay, Cancelled, WeatherDelay.
- Kaggle flight delay datasets — search 'flight delay dataset' on kaggle.com. Several pre-joined datasets exist with weather columns included. This is the fastest path for a project with a time constraint. Recommended: '2015 Flight Delays and Cancellations' or '2019 Airline Delay and Cancellation' datasets.
- OpenSky Network — https://opensky-network.org/datasets — global flight state data if you want non-US airports.

**Scope Decision**

Pick two to four airports. Variety matters: choose at least one large hub (high traffic, multiple runways) and one smaller airport (less infrastructure, more weather-sensitive). Good example pairs: Delhi IGI + Pune (India); Heathrow + Edinburgh (UK); Chicago O'Hare + Memphis (US).

**Deliverable**

Create docs/data_sources.md listing: source name, URL, download date, airport ICAO/IATA codes used, time period covered, number of rows downloaded.

## T-1.2 Raw Data Audit and Data Dictionary

**Goal**

Before touching or transforming anything, thoroughly document what the raw data contains. This becomes the team's shared reference so no one misunderstands a column later.

**Steps**

- Open each raw file in pandas using pd.read_csv(). Run .shape, .dtypes, .head(10), .describe(), and .isnull().sum() on both datasets.
- For every column, record: column name, data type, what it physically means, its unit of measurement (e.g. knots, km, Celsius, minutes), the range of observed values, and the percentage of nulls.
- Note any columns that appear in one dataset but not the other and decide whether they are needed.
- Identify the timestamp column in each dataset and confirm they can be aligned (same timezone? same granularity?).

**Deliverable**

Create docs/feature_dictionary.md as a markdown table. Share this with the full team before Milestone 2 begins. Column names used here must remain frozen — no casual renaming after this point.

## T-1.3 Exploratory Data Analysis (EDA) Notebook

**Goal**

Understand the statistical properties of the data and identify patterns worth highlighting in the dashboard. This notebook is for understanding only — it does not go into production code.

**File**

`notebooks/01_eda.ipynb`

**Analyses to Include**

- Histograms of all weather variables — wind speed, visibility, temperature, rainfall, humidity. Use matplotlib or seaborn.
- Time series chart of delay frequency per airport — line chart by week or month to spot seasonal patterns.
- Correlation matrix between weather features and the delay column. Use seaborn.heatmap(df.corr()).
- Null value heatmap — seaborn.heatmap(df.isnull(), cbar=False, yticklabels=False) to visually confirm which fields have gaps.
- Disruption rate breakdown by airport — a bar chart showing % of flights delayed > 15 minutes per airport. This becomes a key dashboard KPI.
- Delay distribution by hour of day and day of week — heatmap showing when disruptions are most frequent.
- Weather event frequency — how often does visibility drop below 3 km? How often does wind exceed 50 knots? These thresholds feed directly into feature engineering.

**Libraries**

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
```

**Deliverable**

A clean, commented notebook. Each cell should have a markdown explanation of what you found, not just the code. Export one key chart as assets/eda_key_chart.png for the presentation.

### T-1.4 Baseline Data Cleaning Script

**Goal**

Write a reusable, importable Python module that transforms raw CSVs into a single clean dataframe. This is the most critical single file in the project. It must be a proper .py script with named functions, not a notebook.

**File**

`src/data_prep.py`

**Functions to Write**

• load_weather(filepath: str) -> pd.DataFrame — reads the weather CSV, parses the timestamp column into datetime, renames columns to snake_case, filters to the chosen airports only.

• load_flights(filepath: str) -> pd.DataFrame — reads the flight CSV, standardises the airport code column name to 'airport_code', parses FlightDate to datetime, keeps only: FlightDate, airport_code, DepDelay, Cancelled, WeatherDelay.

• merge_datasets(weather_df, flights_df) -> pd.DataFrame — performs a time-windowed merge using pd.merge_asof() sorting both datasets by timestamp and matching each flight to the nearest weather reading within a 1-hour tolerance. Key argument: tolerance=pd.Timedelta('1h'), direction='nearest'.

• handle_missing(df) -> pd.DataFrame — fills numeric weather columns forward in time within each airport group using .groupby('airport_code').ffill(), then fills any remaining nulls with the column median. Drops rows where the target label (DepDelay) is null.

• clean_labels(df) -> pd.DataFrame — adds binary column 'disruption': 1 if DepDelay > 15 (the standard FAA/ICAO threshold for a reportable delay), else 0. Also adds 'delay_minutes' as a clean numeric column capped at 300 minutes to remove outliers.

• run_pipeline(weather_path, flights_path, output_path) -> None — orchestrates all the above in order and saves the result to data/processed/clean_data.csv.

**Critical Constraint**

Do NOT use absolute file paths. Use pathlib.Path(__file__).parent.parent / 'data' / 'raw' / 'filename.csv' so the script works on any team member's machine.

**Reference**

pandas merge_asof documentation: https://pandas.pydata.org/docs/reference/api/pandas.merge_asof.html

## Milestone 2 — Core Build

## T-1.5 Feature Engineering Script

**Goal**

Transform the clean merged dataframe into a rich feature matrix that the ML model can learn from. This is the most intellectually demanding task for Person 1 and is what separates a basic project from an impressive one.

**File**

`src/features.py`

**Function Signature**

```
def build_features(df: pd.DataFrame) -> pd.DataFrame:
```

**Weather-Based Features to Create**

• wind_category — bin wind speed into 4 ordered categories: calm (0-10 knots), moderate (10-25 knots), strong (25-50 knots), severe (50+ knots). Use pd.cut(df['wind_speed'], bins=[0,10,25,50,999], labels=['calm','moderate','strong','severe']).

• visibility_risk — binary flag: 1 if visibility < 3 km (ICAO Category I minimum). This is one of the strongest predictors of disruption.

• rain_severity — bin precipitation into: none (0mm), light (0-2.5mm/hr), moderate (2.5-7.5mm/hr), heavy (7.5+mm/hr).

• temp_stress — binary: 1 if temperature > 35°C (heat stress on equipment and tarmac) or < -10°C (ice risk, de-icing delays).

• storm_flag — composite binary: 1 if (wind_speed > 50 knots) OR (visibility < 1 km) OR (rain_severity == 'heavy'). This is the highest-level weather danger indicator.

• humidity_stress — binary: 1 if relative humidity > 90% AND temperature > 28°C (combined heat + humidity fog risk).

**Time-Based Features**

• hour_of_day, day_of_week, month — extracted from the datetime column using .dt.hour, .dt.dayofweek, .dt.month.

• is_weekend — binary: 1 if day_of_week >= 5.

• is_peak_hour — binary: 1 if hour_of_day is between 6-9 or 16-20 (typical peak departure/arrival windows).

• is_holiday_season — binary: 1 if month is in [3, 10, 11, 12] (Diwali, Christmas, Eid travel surges depending on your chosen region).

**Operational History Features (Rolling)**

• delay_last_1h — average disruption value over the previous 1 hour for the same airport. Use df.groupby('airport_code')['disruption'].transform(lambda x: x.shift(1).rolling('1H', min_periods=1).mean()).

• delay_last_3h — same but over 3 hours. This captures cascading delay effects.

• delay_last_24h — rolling 24-hour delay rate. Captures chronic congestion or weather patterns.

• congestion_level — rolling count of flights in the past 2 hours per airport. High congestion amplifies weather-related disruption.

**Important Leakage Warning**

Every rolling/lag feature MUST use .shift(1) before the rolling window. Never include the current row in its own feature. Data leakage is the most common mistake in time-series ML and will make your model look better than it is while being useless in production.

**Deliverable**

Update docs/feature_dictionary.md with every new feature: name, formula, rationale. Save the feature matrix as data/processed/features.csv.

## T-1.6 KPI Design for Dashboard

**Goal**

Define the exact business metrics that the dashboard will display. This is where the data analytics dimension of the project becomes visible. Person 3 cannot build meaningful visualisations without these definitions.

**Deliverable File**

`docs/kpi_definitions.md`

**KPIs to Define**

- Disruption Rate (%) — percentage of flights with disruption=1 in a selected time window and airport. Formula: sum(disruption) / count(disruption) * 100.
- Average Recovery Time (minutes) — mean of delay_minutes where disruption=1. Tells operations how long recovery typically takes once a disruption starts.
- Weather Sensitivity Score — Pearson correlation coefficient between storm_flag and disruption for a given airport, computed over the training period. Higher = more weather-vulnerable.
- Current Risk Level — output of the ML model: Low (<30% probability), Medium (30–65%), High (>65%). This is the primary live indicator on the dashboard.
- Resilience Index — the 0–100 score computed by Person 2. Higher = more resilient.
- Peak Hour Disruption Premium — ratio of disruption rate during peak hours vs off-peak. A value of 2.0 means disruptions are twice as likely during peak hours.

**Format**

For each KPI: write the plain English definition, the formula, the data columns it uses, the unit, and whether it is computed at query time or pre-computed. Person 3 needs all of this to build the dashboard widgets correctly.

### T-1.7 Resilience Score Input Dataset

**Goal**

Produce a per-airport aggregated summary dataframe that Person 2 will use to compute the resilience index. This must be ready before Person 2 starts T-2.6.

**Function**

```
def build_resilience_inputs(df: pd.DataFrame) -> pd.DataFrame: # in src/resilience.py
```

**Output Columns (one row per airport)**

- airport_code — IATA/ICAO identifier.
- disruption_rate — % of flights delayed across the full dataset period.
- avg_recovery_time — mean delay_minutes where disruption=1.
- weather_sensitivity — correlation(storm_flag, disruption) across all rows for this airport.
- congestion_sensitivity — correlation(congestion_level, disruption).
- peak_hour_disruption_rate — disruption rate restricted to is_peak_hour=1 rows only.
- data_period_days — number of calendar days covered by this airport's data (for transparency).

**Output File**

```
data/processed/airport_resilience_inputs.csv
```

**Note**

Include a brief comment in the function explaining what each column measures. Person 2 and Person 4 both need to understand these columns to document them correctly.

## Milestone 3 — Integration & Quality

T-1.8

## Final Data Quality Report

**Goal**

Perform a final audit of the entire data pipeline before the project is submitted. This ensures the model was trained on legitimate data and the dashboard metrics are correct.

**Checks to Perform**

- Data leakage check — confirm every rolling/lag feature uses shift(1). Load the features CSV and verify the first row of each rolling column is NaN (expected, because there is no prior row).
- Label integrity — confirm the disruption column has zero nulls: assert df['disruption'].isnull().sum() == 0.
- Train/test split correctness — confirm the split is time-based, not random. The test set must contain only dates after the training set. Print: 'Train period: {min} to {max}. Test period: {min} to {max}.' Confirm they do not overlap.
- Class balance check — print the disruption rate in the full dataset. If disruptions are fewer than 10% of rows, flag this for Person 2 (they may need class weighting or oversampling).
- Airport coverage check — confirm all chosen airports appear in the final features CSV and that each has at least 500 rows.
- Column name consistency — confirm that the column names in features.csv exactly match what is documented in docs/feature_dictionary.md. Any mismatch breaks Person 2's model.

**Deliverable**

`docs/data_quality_report.md`

One paragraph per check. State the result (pass/fail) and any action taken. This document goes into the appendix of the final report.

## 3. Person 2 — ML & Explainability Lead

Person 2 is responsible for the entire machine learning and explainability stack. This is the technical core of the CLEARS system. The work must be written as clean, importable Python modules with well-defined function signatures — not as standalone notebooks — because Person 3's dashboard calls these functions directly. Every model must be saved to disk so the dashboard can load it without re-training.

**Milestone 1 — Foundation**

### T-2.1 Baseline Model Notebook

**Goal**

Before the full clean dataset is ready, prove the ML pipeline works end to end using a minimal version of the data. This is a validation step, not a final model.

**File**

`notebooks/02_baseline_model.ipynb`

**Steps**

- Load a partial version of the dataset from Person 1 (even if it is just weather columns + the disruption label with no engineered features).
- Split using train_test_split with shuffle=False (time-ordered split, even at this stage).
- Train a LogisticRegression and a DecisionTreeClassifier from sklearn.
- Evaluate both using: accuracy_score, classification_report (precision, recall, F1 per class), roc_auc_score, and a confusion matrix plotted with ConfusionMatrixDisplay.
- Print a feature importance chart from the DecisionTreeClassifier using plt.barh().

**Purpose**

This step exists to catch pipeline errors early. If the merge or labelling is wrong, you will see it here. It also gives you a baseline F1 to beat with XGBoost later.

**Reference**

sklearn model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html

### T-2.2 Define Model Interface Stubs

**Goal**

Create the prediction module immediately so Person 3 can start building the dashboard UI without waiting for the real model. Stubs return hardcoded fake values and will be replaced in Milestone 2.

**File**

`src/predict.py`

**Stub Functions to Write Now**

```
def predict_disruption(feature_dict: dict) -> dict:
return {'disruption': 0, 'probability': 0.42}
def predict_severity(feature_dict: dict) -> dict:
return {'severity': 'low', 'severity_code': 0}
```

**Why This Matters**

Person 3 imports these functions into streamlit_app.py on Day 1. Without stubs, the dashboard cannot be built until the model is trained. With stubs, both Person 2 and Person 3 work in parallel and the final merge is just swapping in the real model.

**Interface Contract**

The feature_dict argument must use exactly the column names from docs/feature_dictionary.md. Do NOT change the function signatures later — that would break Person 3's code.

## Milestone 2 — Core Build

### T-2.3 Train the Main Disruption Prediction Model

**Goal**

Build, evaluate, and save the primary binary classification model that predicts whether a weather event will cause airport disruption.

**File**

`src/model_train.py`

**Model Choice**

Use XGBClassifier from the xgboost library or LGBMClassifier from lightgbm. Both handle tabular structured data extremely well and produce feature importances natively. XGBoost is slightly more familiar; LightGBM trains faster on large datasets. Either is acceptable.

**Training Steps**

- Load data/processed/features.csv. Drop non-feature columns: airport_code, timestamp, delay_minutes (raw target — not a feature).
- Time-based train/test split: use the last 20% of dates as the test set. Sort by date first. Do NOT use random splitting — that would leak future information into training.
- Handle class imbalance: pass scale_pos_weight = (count of 0s) / (count of 1s) to XGBClassifier. This compensates for the fact that delays are less frequent than normal operations.
- Fit the model: model.fit(X_train, y_train, eval_set=[(X_test, y_test)], early_stopping_rounds=30).
- Evaluate: print classification_report, roc_auc_score, and a confusion matrix. Target: F1 >= 0.70 on the minority class (disruption=1). If below this, tune hyperparameters.
- Hyperparameter tuning: use sklearn GridSearchCV on a small grid: n_estimators=[100,300], max_depth=[3,5,7], learning_rate=[0.05,0.1]. Or use Optuna for a more thorough search (pip install optuna).

**Saving the Model**

```
import joblib
joblib.dump(model, 'data/models/disruption_model.pkl')
```

**Update predict.py**

Replace the stub with: load the pkl once at module level using joblib.load(). In predict_disruption(), build a DataFrame from feature_dict and call model.predict_proba().

## T-2.4 Delay Severity Estimation Model

**Goal**

Build a second model that goes beyond yes/no disruption and predicts how severe the delay will be. This makes the project significantly more useful to operations teams.

**Label Creation**

Create a severity_label column by binning delay_minutes: 0 = no disruption (delay_minutes < 15), 1 = minor (15–45 min), 2 = moderate (45–120 min), 3 = severe (120+ min). Use pd.cut() with include_lowest=True.

```
bins = [0, 15, 45, 120, 9999]
labels = [0, 1, 2, 3]
df['severity_label'] = pd.cut(df['delay_minutes'], bins=bins, labels=labels)
```

**Model**

Train a second XGBClassifier with objective='multi:softprob' and num_class=4. Use the same train/test time split as T-2.3. Evaluate with a multiclass classification_report.

**Saving**

```
joblib.dump(severity_model, 'data/models/severity_model.pkl')
```

**Update predict.py**

Update predict_severity() to load severity_model.pkl and map the predicted class integer back to a human-readable string: {0:'none', 1:'minor', 2:'moderate', 3:'severe'}. Return both the string label and the class probabilities so the dashboard can show confidence.

## T-2.5 SHAP Explainability Module

**Goal**

Make the model's predictions interpretable so that operations staff and project evaluators understand WHY a disruption is predicted, not just that it is. This is the most impressive technical differentiator of the project.

**File**

`src/explain.py`

**Setup**

```
pip install shap
import shap
explainer = shap.TreeExplainer(model) # works for both XGB and LGBM
```

**Function 1: Single-Prediction Explanation**

```
def explain_prediction(model, feature_dict: dict) -> dict:
```

Build a one-row DataFrame from feature_dict. Run explainer.shap_values(X_single). Return a dict mapping each feature name to its SHAP value for this prediction. Positive SHAP = pushed prediction toward disruption. Negative = pushed toward no disruption.

**Function 2: Global Feature Importance**

```
def get_global_importance(model, X_test: pd.DataFrame) -> pd.Series:
```

Run explainer.shap_values(X_test). Compute mean(abs(shap_values)) across all rows. Return as a pd.Series sorted descending. This is what the dashboard's feature importance bar chart displays.

**Static Plot**

Generate a SHAP beeswarm summary plot using shap.summary_plot(shap_values, X_test, show=False). Save with plt.savefig('assets/shap_summary.png', bbox_inches='tight', dpi=150). This image goes into the presentation slides.

**Function 3: Scenario Risk**

```
def scenario_risk(base_features: dict, overrides: dict) -> dict:
```

Merge base_features with overrides (overrides take priority). Call predict_disruption() with the merged dict. Return {'base_probability': float, 'scenario_probability': float, 'delta': float}. This powers the dashboard's scenario simulator.

**Reference**

SHAP TreeExplainer tutorial: https://shap.readthedocs.io/en/latest/example_notebooks/tabular_examples/tree_based_models/

## T-2.6 Resilience Index Computation

**Goal**

Compute a single interpretable resilience score (0–100) per airport that summarises how well each airport handles weather stress. This is one of the most unique contributions of the CLEARS project.

**File**

`src/resilience.py — add function compute_resilience_index()`

**Input**

Load data/processed/airport_resilience_inputs.csv (produced by Person 1 in T-1.7).

**Algorithm**

- Step 1: Min-max normalise each input column to [0, 1] across airports: (value - min) / (max - min).
- Step 2: Compute a weighted vulnerability score per airport. Suggested weights: disruption_rate = 0.35, avg_recovery_time = 0.30, weather_sensitivity = 0.20, congestion_sensitivity = 0.15.
- Step 3: The resilience index = 100 - (vulnerability_score * 100). Higher score = more resilient. An airport with zero disruption history and fast recovery scores 100.
- Step 4: Add a qualitative tier: 0–40 = 'Vulnerable', 41–70 = 'Moderate', 71–100 = 'Resilient'.

**Output**

`data/processed/resilience_scores.csv — columns: airport_code, resilience_index, tier`

**Note on Weights**

The weights are a design choice. Document your reasoning in docs/model_report.md. You can argue that recovery time matters more than disruption frequency for operational planning — that is a valid and defensible position.

### T-2.7 Model Evaluation Report

**Goal**

Produce a clear written record of model performance that goes into the final report and presentation. This proves you did rigorous evaluation, not just 'it works'.

**File**

`docs/model_report.md`

**Sections to Include**

- Model architecture — algorithm name, library version, key hyperparameters used.
- Training data size — number of rows, airports, time period, class balance ratio.
- Test metrics — accuracy, precision, recall, F1 (per class), ROC-AUC. Include the confusion matrix as a table.
- Comparison to baseline — how much better is XGBoost vs the logistic regression from T-2.1? State the F1 improvement clearly.
- Top 10 SHAP features — list the ten most important features by mean absolute SHAP value. For each, state what it measures and why it intuitively makes sense.
- Known limitations — e.g., limited airport coverage, no real-time weather feed, potential geographic bias if trained only on one region.
- Resilience index methodology — the formula, the weights, and the reasoning behind the weight choices.

**Milestone 3 — Integration**

### T-2.8 Final Model Verification

**Goal**

Before submission, verify the saved models load correctly and produce sensible predictions. This catches any serialisation or environment issues before the demo.

**Checks to Run**

- Fresh load test — in a clean Python session (restart kernel), import predict.py and call predict_disruption({'wind_category': 'severe', 'visibility_risk': 1, 'storm_flag': 1, 'is_peak_hour': 1, ...}). Confirm probability is high (>0.60).
- Sanity check reverse — call with all benign values (calm wind, good visibility, no rain, off-peak). Confirm probability is low (<0.30).
- Severity consistency — confirm that when predict_disruption returns high probability, predict_severity also returns 'moderate' or 'severe', not 'none'.
- SHAP consistency — confirm explain_prediction returns a dict with the correct feature names matching docs/feature_dictionary.md exactly.
- Scenario function — call scenario_risk with a severe weather override and confirm scenario_probability > base_probability.

**Deliverable**

A short verification log in docs/model_verification.md: each check, the input, the output, and pass/fail. Person 4 will format this for the final report appendix.

## 4. Person 3 — App & Dashboard Lead

Person 3 converts all the technical work from Persons 1 and 2 into a polished, interactive product that non-technical evaluators can immediately understand. Streamlit is the chosen framework because it allows rapid Python-native dashboard development without front-end knowledge. The dashboard must be visually clear, well-labelled, and demoable in under five minutes.

### Milestone 1 — Foundation

#### T-3.1 Dashboard Layout Sketch

**Goal**

Plan the full dashboard layout before writing a single line of code. Share the plan with the team so Person 1 knows what metrics to compute and Person 2 knows what the model output needs to look like.

**Panels to Plan**

- Header row: airport selector dropdown, date range picker, current risk badge (colour-coded: green/amber/red).
- Panel 1: Disruption risk gauge or KPI scorecard showing current probability, severity level, and resilience index for the selected airport.
- Panel 2: Weather trend chart — time series of disruption probability over the selected date range.
- Panel 3: SHAP feature importance bar chart — horizontal bars showing which factors drove the current risk level.
- Panel 4: Airport resilience ranking table — all airports sorted by resilience index with tier badge.
- Panel 5: Scenario simulator — sliders for weather variables, side-by-side risk comparison before/after.
- Panel 6: Recommendation card — bullet list of suggested operational actions based on current risk.

**Tool Options**

Sketch on paper, in Figma (free tier at figma.com), in Google Slides, or in Excalidraw (excalidraw.com — free, no login). Share the sketch in the team group before starting T-3.2.

**Questions to Answer in the Sketch**

Which panels appear above the fold? Which are in tabs vs scrolled? What is the sidebar content? What filters apply globally vs per-panel?

### T-3.2 Streamlit Skeleton App

**Goal**

Get a working (but empty) Streamlit app running on every team member's machine. This proves the environment is set up correctly before complex logic is added.

**File**

`app/streamlit_app.py`

**Install**

```
pip install streamlit plotly
```

**Code Structure**

- st.set_page_config(page_title='CLEARS Dashboard', layout='wide') — must be the first Streamlit call.
- Sidebar: st.sidebar.selectbox for airport, st.sidebar.date_input for date range.
- Main area: st.columns([1,1,1]) for the top three KPI cards. Each card is an st.metric().
- Below: st.tabs(['Risk Overview', 'Scenario Simulator', 'Resilience Rankings', 'Recommendations']) — use tabs to organise the six panels without overwhelming a single page.
- Import from src/predict.py using stub functions (T-2.2). Display dummy values in each metric.

**Run Command**

```
streamlit run app/streamlit_app.py
```

**Resource**

Streamlit API reference: https://docs.streamlit.io/library/api-reference

## Milestone 2 — Core Build

## T-3.3 Connect Model Outputs to Dashboard

**Goal**

Replace every dummy value with real calls to Person 2's trained model functions. This is the integration step that makes the dashboard live.

**Steps**

- At the top of streamlit_app.py: import predict, explain, recommend from the src/ directory. Use sys.path.insert(0, str(Path(__file__).parent.parent)) to make the imports work regardless of where streamlit is launched from.
- Build a feature_dict from the sidebar selections and the most recent processed data row for the selected airport and date. This dict must contain all the column names from docs/feature_dictionary.md.
- Call predict.predict_disruption(feature_dict) and display: probability as st.metric('Disruption Risk', f'{prob:.0%}'), disruption label as st.metric('Status', 'At Risk' if prob > 0.5 else 'Normal').
- Call predict.predict_severity(feature_dict) and display the severity label with a colour-coded st.warning() or st.error() or st.success() depending on severity code.
- Call explain.get_global_importance() once at app startup (cached with @st.cache_data) and display as a horizontal bar chart using plotly.express.bar(orientation='h').
- Load data/processed/resilience_scores.csv and display in st.dataframe() with conditional formatting: colour the resilience_index column using a green-to-red scale.

**Caching**

Wrap any function that loads files or runs model inference in @st.cache_data so that Streamlit does not re-run them on every widget interaction. This is critical for dashboard responsiveness.

## T-3.4 Weather Trend Chart

**Goal**

Show the user how disruption risk has varied over time for the selected airport. This gives context to the current risk value and reveals seasonal or time-of-day patterns.

**Data**

Load data/processed/features.csv. Filter to the selected airport and date range using the sidebar inputs. Compute a daily rolling average of the disruption column.

**Chart**

```
import plotly.express as px
fig = px.line(daily_df, x='date', y='disruption_rate',
color='airport_code', title='Disruption Rate Over Time')
fig.update_traces(line_width=2)
st.plotly_chart(fig, use_container_width=True)
```

**Enhancements**

- Add a second line showing the 7-day rolling average to smooth noise.
- Add vertical dashed lines for any storm_flag=1 days using fig.add_vline().
- Add a shaded band showing the model's prediction interval if available.

**Label Everything**

Every chart axis must have a clear label. Use fig.update_layout(xaxis_title='Date', yaxis_title='Disruption Rate (%)'). Non-technical evaluators must be able to understand every chart without explanation.

T-3.5

## Scenario Simulator UI

**Goal**

Allow users to interactively test 'what if' weather scenarios and see how predicted disruption risk changes. This is the most impressive interactive feature of the dashboard.

**Layout**

Place inside the 'Scenario Simulator' tab. Use two columns: left column for sliders/inputs, right column for the results.

**Input Widgets**

- st.slider('Visibility (km)', min_value=0.0, max_value=10.0, value=5.0, step=0.5) — maps to visibility_risk: 1 if < 3.
- st.slider('Wind Speed (knots)', min_value=0, max_value=100, value=15, step=5) — maps to wind_category.
- st.selectbox('Rainfall Intensity', options=['none','light','moderate','heavy']) — maps to rain_severity.
- st.slider('Temperature (°C)', min_value=-20, max_value=50, value=25, step=1) — maps to temp_stress.
- st.checkbox('Peak Hour Traffic') — maps to is_peak_hour.

**Calling the Scenario Function**

```
overrides = {'visibility_risk': 1 if vis < 3 else 0,
'wind_category': wind_cat, 'rain_severity': rain,
'temp_stress': 1 if temp > 35 or temp < -10 else 0,
'is_peak_hour': int(peak)}
result = explain.scenario_risk(base_features, overrides)
```

**Displaying Results**

Use st.metric() with the delta argument to show the change: st.metric('Scenario Risk', f"{result['scenario_probability']:.0%}", delta=f"{result['delta']:+.0%}"). A red delta means increased risk. This is immediately readable.

**Preset Scenarios**

Add three preset buttons: 'Severe Storm', 'Light Rain + Peak Hour', 'Clear Day'. Each button sets the sliders to a predefined combination using st.session_state.

### T-3.6 Recommendation Engine and Display

**Goal**

Present actionable operational guidance based on the current risk level and top SHAP features. This is what transforms CLEARS from a prediction tool into a decision-support system.

**File**

`src/recommend.py`

**Function**

```
def get_recommendation(prob: float, severity: str, top_features: list) -> list[str]:
```

**Rule Logic**

- If prob > 0.80: add 'Initiate proactive departure delay programme', 'Alert all gate agents of imminent disruption', 'Pre-position ground crew for recovery operations', 'Notify inbound airlines of potential slot disruptions'.
- If 0.50 < prob <= 0.80: add 'Monitor weather feed every 15 minutes', 'Prepare staffing buffers for departure terminals', 'Review inbound aircraft positions and estimated arrival times'.
- If prob <= 0.50: add 'Conditions nominal — continue standard operations', 'Schedule recommended: no proactive action required'.
- If 'visibility_risk' is in top_features: add 'Coordinate low-visibility procedures with ATC — ILS Category II/III may be required'.
- If 'wind_category' is in top_features: add 'Review crosswind component limits for active runways — consider runway change if sustained gusts exceed 25kt'.
- If 'storm_flag' is in top_features: add 'Issue SIGMET advisory to connected airlines if storm flag active'.
- If 'congestion_level' is in top_features: add 'Implement ground delay programme to reduce airborne stack congestion'.

**Display in Dashboard**

Call get_recommendation() and display each item as a separate st.info(), st.warning(), or st.error() depending on urgency. High-risk actions use st.error (red). Monitoring actions use st.warning (amber). Nominal actions use st.info (blue). This colour coding is immediately readable.

### T-3.7 Resilience Heatmap / Airport Comparison Visual

**Goal**

Provide a visual comparison of resilience scores across airports. This is the analytics showpiece of the dashboard and makes the resilience index tangible.

**Option A: Bar Chart (simpler, always works)**

```
fig = px.bar(resilience_df.sort_values('resilience_index'),
x='resilience_index', y='airport_code',
color='tier', orientation='h',
color_discrete_map={'Resilient':'#1D9E75','Moderate':'#EF9F27','Vulnerable':'#D85A30'},
title='Airport Resilience Ranking')
```

**Option B: Scatter Map (more impressive, requires coordinates)**

Add latitude and longitude columns to resilience_scores.csv manually for each airport. Then use plotly.express.scatter_geo() with color='resilience_index' and color_continuous_scale='RdYlGn'. This puts the airports on a world map with colour showing their resilience level.

```
fig = px.scatter_geo(resilience_df, lat='lat', lon='lon',
color='resilience_index', size='disruption_rate',
hover_name='airport_code',
color_continuous_scale='RdYlGn',
title='Global Airport Resilience Map')
```

**Recommendation**

Build Option A first (guaranteed to work). Add Option B as an enhancement if time allows. Both can coexist in the same tab using st.columns.

## Milestone 3 — Polish & Demo

### T-3.8 UI Polish and Demo Script

**Goal**

Ensure the dashboard is visually clean, responsive, and ready to demo in front of evaluators. Write a step-by-step demo script so every team member can present it identically.

**UI Polish Steps**

- Add st.spinner('Running prediction...') around model inference calls so the UI does not freeze silently.
- Add help='...' tooltips to every slider and dropdown so non-technical users understand what each control does. Example: st.slider('Visibility (km)', help='Aviation Category I minimum is 3 km. Below this, ILS approaches may be required.').
- Add a project branding header: use st.image('assets/clears_logo.png') if a logo exists, otherwise use st.title('CLEARS — Airport Resilience Dashboard') with a subtitle st.caption().
- Confirm all charts have axis labels, chart titles, and legends. No unlabelled axes allowed.
- Test on a fresh machine (or fresh Python environment) to confirm there are no import errors or missing file path issues.

**Demo Script**

`docs/demo_script.md`

- Step 1 (30s): Open dashboard. Show header, select Airport A, set date range to last 3 months. Point to the risk gauge and resilience index.
- Step 2 (60s): Go to Risk Overview tab. Explain the trend chart — highlight a spike that corresponds to a known weather event. Explain the SHAP bar chart: 'This shows that visibility and wind were the primary drivers on that day.'
- Step 3 (60s): Go to Scenario Simulator tab. Set visibility to 1 km and rainfall to heavy. Show the risk jump from X% to Y%. Explain: 'This is the what-if analysis — it lets operations teams prepare before conditions worsen.'
- Step 4 (30s): Go to Recommendations tab. Read two recommendations aloud. Explain they are driven by the scenario inputs, not hardcoded.
- Step 5 (30s): Go to Resilience Rankings tab. Point to the most vulnerable airport and explain what made it score low.

**Total Demo Time**

Target: under 4 minutes for the core demo. Allocate 1 minute for questions. Practice this sequence at least twice before the presentation.

## 5. Person 4 — Documentation & Support Lead

Person 4 keeps the project organised, documented, and submission-ready. This role requires precision, consistency, and proactive communication rather than deep technical knowledge. Every deliverable from Person 4 directly improves the perceived quality of the project. Poor documentation can undermine even technically excellent work. Good documentation makes a mediocre model look professional.

**Milestone 1 — Foundation**

### T-4.1 GitHub Repository Setup and Folder Structure

**Goal**

Create the shared repository and establish the folder structure that every team member will use from Day 1. Do this before anyone writes a single line of code.

**Repository Setup Steps**

- Create a new GitHub repository named 'CLEARS' under one team member's account. Set visibility to Private. Add all team members as collaborators (Settings > Collaborators).
- Protect the main branch: Settings > Branches > Add Rule > check 'Require a pull request before merging'. This prevents anyone from accidentally pushing directly to main.
- Create the following folders by adding a .gitkeep file inside each (GitHub requires at least one file to show a folder):

```
data/raw/ data/processed/ data/models/
notebooks/ src/ app/
assets/ docs/ tests/
```

**Files to Create Immediately**

- README.md — see T-4.1 continued below.
- .gitignore — must include: data/raw/*.csv, data/raw/*.zip, *.pkl (large model files should be excluded from git and shared via Google Drive), __pycache__/, .ipynb_checkpoints/, .env, *.pyc.
- requirements.txt — see T-4.2 for full contents.
- docs/CONTRIBUTING.md — branch naming convention: use 'p1/feature-name', 'p2/feature-name' etc. so it is clear who owns each branch.

**README.md Content**

Include: project name and acronym expansion, one paragraph description, team member names and roles, setup instructions (clone, create venv, pip install -r requirements.txt, run the app), folder structure diagram, and a link to docs/data_sources.md.

### T-4.2 requirements.txt and Environment Management

**Goal**

Ensure every team member uses identical library versions. Version conflicts are the single most common cause of 'it works on my machine' failures in group projects.

**Initial requirements.txt**

```
pandas>=2.0.0
numpy>=1.24.0
scikit-learn>=1.3.0
xgboost>=2.0.0
lightgbm>=4.0.0
shap>=0.43.0
streamlit>=1.28.0
plotly>=5.17.0
matplotlib>=3.7.0
seaborn>=0.12.0
joblib>=1.3.0
openpyxl>=3.1.0
optuna>=3.3.0
pytest>=7.4.0
```

**Environment Instructions in README**

```
python -m venv clears_env
source clears_env/bin/activate # Mac/Linux
clears_env\Scripts\activate # Windows
pip install -r requirements.txt
```

**Python Version**

Agree on Python 3.10 or 3.11 as a team. Document this in README.md. If team members have different versions, use pyenv to manage multiple Python versions on one machine.

**Maintenance Responsibility**

Person 4 owns requirements.txt. Whenever any team member installs a new package, they must tell Person 4, who updates the file and commits the change. No one else edits this file.

### T-4.3 Presentation Skeleton

**Goal**

Create the slide deck structure so that content can be filled in progressively during Milestones 2 and 3 rather than scrambled together the night before submission.

**Slide Order**

- Slide 1 — Title: CLEARS, project subtitle, team member names, institution name, date.
- Slide 2 — Problem Statement: one slide, three bullet points. What problem exists? Why does it matter? What is currently missing?
- Slide 3 — Project Overview: the pipeline diagram from this document as the visual. One sentence per layer.
- Slide 4 — Data: what datasets were used, airports chosen, time period, row counts. Include a simple map showing airport locations.
- Slide 5 — Feature Engineering: table of key engineered features (name, description). Highlight the most novel ones.
- Slide 6 — Model Architecture: algorithm chosen, why, key hyperparameters. One sentence on the severity model.
- Slide 7 — Results: classification report table, ROC-AUC score, comparison to baseline. Use a visual confusion matrix.
- Slide 8 — Explainability: the SHAP beeswarm plot from assets/shap_summary.png. Caption: top 5 features and their direction of influence.
- Slide 9 — Resilience Index: bar chart of airport resilience scores. Explain one low-scoring airport in detail.
- Slide 10 — Dashboard Demo: screenshots of each dashboard tab. Annotate each screenshot with arrows pointing to key features.
- Slide 11 — Scenario Analysis: a before/after comparison for a specific storm scenario. Show the risk increase numerically.
- Slide 12 — Conclusion: what the project achieved, what it could not do, and one future enhancement per team member.

**Tool**

Google Slides (shareable link, real-time collaboration) or PowerPoint. Create placeholders (grey boxes) for every chart and screenshot — these are filled in during Milestone 3.

## Milestone 2 — Core Build

### T-4.4 Label Verification and Dataset Sanity Checking

**Goal**

Manually verify that Person 1's disruption label column is correctly computed. Even small labelling errors (e.g., a wrong threshold, an inverted condition) can silently corrupt the model's learning.

**Steps**

- Load data/processed/clean_data.csv. Filter to 50 random rows where disruption=1 and 50 rows where disruption=0.
- For each row: check whether the delay_minutes value is consistent with the disruption label. If disruption=1, delay_minutes should be >15. If disruption=0, delay_minutes should be <=15. Any row that does not match is a labelling error.
- Check the storm_flag column: whenever storm_flag=1, does the underlying weather data (visibility, wind, rain) support it? Spot-check at least 20 rows.
- Check rolling features: for 5 rows, manually compute delay_last_1h by looking at the previous rows for the same airport. Confirm the value in the column matches your manual calculation.

**Reporting**

Create docs/label_verification.md. State: number of rows checked, number of discrepancies found, and whether each was fixed or accepted. If you find errors, notify Person 1 immediately.

**What to Avoid**

Do not modify the data yourself. If you find errors, document them and ask Person 1 to fix the source function in data_prep.py. Data files must only be regenerated by the person who owns the cleaning script.

## T-4.5 Test Cases for Model Functions

**Goal**

Write automated sanity checks that can be run before every merge to confirm the model functions behave correctly. These are not full unit tests — they are smoke tests that catch obvious breakage.

**File**

`tests/test_predict.py`

**Tests to Write**

- test_predict_disruption_returns_correct_keys(): call predict_disruption({...}) with a complete feature dict. Assert the return value is a dict with keys 'disruption' and 'probability'. Assert 0 <= probability <= 1.
- test_predict_disruption_high_risk(): call with storm_flag=1, visibility_risk=1, wind_category='severe', is_peak_hour=1. Assert probability > 0.5.
- test_predict_disruption_low_risk(): call with all benign values. Assert probability < 0.5.
- test_predict_severity_returns_valid_label(): call predict_severity({...}). Assert severity is one of ['none','minor','moderate','severe'].
- test_scenario_risk_increases_with_storm(): call scenario_risk(base_features, {'storm_flag': 1, 'visibility_risk': 1}). Assert scenario_probability > base_probability.
- test_get_recommendation_returns_list(): call get_recommendation(0.85, 'severe', ['visibility_risk', 'wind_category']). Assert the return value is a non-empty list of strings.

**Running Tests**

```
pytest tests/ -v
```

**Timing**

These tests should be run by Person 4 after every major merge. If any test fails, block the merge and notify the responsible person.

### T-4.6 Documentation Review and Consistency Audit

**Goal**

Review all documentation files for clarity, consistent terminology, correct formatting, and agreement with the actual code. Inconsistent docs are one of the most common evaluation deductions in group projects.

**Files to Review**

- docs/data_sources.md — confirm all URLs are live, all download dates are filled in, all airports are listed with both IATA and ICAO codes.
- docs/feature_dictionary.md — confirm every column in data/processed/features.csv has a corresponding entry. Confirm no column name in the dictionary differs (even by one character) from the actual CSV header.
- docs/kpi_definitions.md — confirm the KPI formulas are mathematically correct. Ask Person 1 to verify.
- docs/model_report.md (once Person 2 writes it) — check that metric values reported match what is actually printed by the evaluation code. Ask Person 2 to confirm.

**Consistency Checks**

- Airport names — are they always referred to by the same code (IATA or ICAO)? Pick one and enforce it throughout all docs.
- Threshold values — is the delay threshold always stated as '>15 minutes'? Is the low-visibility threshold always '< 3 km'? Any variation is confusing.
- Person references — ensure all docs say 'Person 1', 'Person 2' etc., not first names (for anonymity in submission if required).

**Deliverable**

Create docs/review_log.md: a table of every file reviewed, date reviewed, issues found, and whether the issue was resolved.

**Milestone 3 — Final Integration**

### T-4.7 Final Report Compilation

**Goal**

Compile all module documentation and findings into a single coherent project report. This is the primary written submission artefact.

**File**

`docs/final_report.md (or .pdf if a formatted submission is required)`

**Report Structure**

- Abstract (150 words max): what the project does, key results, main contribution.
- 1. Introduction: the problem, why airports are weather-sensitive, what existing solutions miss, what CLEARS adds.
- 2. Data: sources, airports chosen, time period, preprocessing steps, final dataset dimensions and class balance.
- 3. Methodology: feature engineering rationale (why each feature was created), model selection rationale (why XGBoost over alternatives), explainability approach (why SHAP), resilience index design (the formula and weight choices).
- 4. Results: model metrics table, confusion matrix, comparison to baseline, top SHAP features with interpretation.
- 5. Resilience Analysis: resilience scores per airport, which airport is most/least resilient and why, a comparison between two airports of contrasting score.
- 6. Dashboard: description of each panel, a screenshot per tab, explanation of the scenario simulator with one example.
- 7. Limitations: data coverage gaps, single weather data source, no real-time feed, geographic restriction, potential class imbalance effects.
- 8. Conclusion: what was achieved, future enhancements (real-time METAR feed integration, multi-runway modelling, airline-specific risk segmentation).
- Appendix A: data quality report (from T-1.8). Appendix B: model verification log (from T-2.8). Appendix C: full feature dictionary.

**Tone**

Write in third person, past tense, with precise language. Avoid vague statements like 'good results'. State exact numbers: 'The XGBoost model achieved an F1 score of 0.74 on the minority class, a 31% improvement over the logistic regression baseline.'

## T-4.8 Presentation Finalisation and Rehearsal Coordination

**Goal**

Complete the slide deck, confirm every team member can speak to their own section, and run at least one full rehearsal before the actual presentation.

**Slide Completion Checklist**

- Every chart placeholder replaced with the actual chart. No placeholder text remaining.
- Slide 7 (Results): model metrics filled in from docs/model_report.md. Confirmed with Person 2.
- Slide 8 (Explainability): assets/shap_summary.png inserted. Top 5 features listed with interpretation bullets.
- Slide 10 (Dashboard): screenshots taken from a working demo session. Annotated with arrows or callout boxes.
- Every slide: font size >= 18pt for body text, >= 24pt for titles. No walls of text — maximum 5 bullet points per slide.
- Speaker notes: every slide has at least one sentence of speaker notes explaining what to say. This is critical if the presenter freezes under pressure.

**Rehearsal Plan**

- Rehearsal 1 (2 days before submission): each person presents their own section in sequence. Time the full run. Target: under 12 minutes for a 4-person team.
- Rehearsal 2 (1 day before submission): Person 3 runs the live dashboard demo while others speak. Confirm the demo sequence from docs/demo_script.md works without errors.
- Identify the one question most likely to be asked by an evaluator ('Why did you choose XGBoost over a neural network?', 'How does the resilience index handle airports with no historical data?') and prepare answers.

**Contingency**

Export the dashboard as screenshots in case of internet or dependency failure on demo day. These screenshots go into the slide deck as a fallback. Never rely solely on a live demo.

## 6. Collaboration Rules & Shared Conventions

| Rule | Detail |
|---|---|
| **Python version** | 3.10 or 3.11. Agree on day one. Document in README. |
| **File paths** | Always relative. Use pathlib.Path(__file__).parent to build paths. No hardcoded absolute paths. |
| **Column names** | Frozen after T-1.2. Any change requires updating feature_dictionary.md and notifying the team. |
| **Branch naming** | p1/task-name, p2/task-name, p3/task-name, p4/task-name. Never work on main directly. |
| **Merging** | Only via pull request. At least one other person must review before merge. |
| **Model files (.pkl)** | Never commit to git — they are large. Share via Google Drive. Document the Drive link in README. |
| **Data files (.csv)** | data/raw/ excluded from git. data/processed/ can be committed if <10MB. Check .gitignore. |
| **requirements.txt** | Only Person 4 edits this file. Everyone else reports new packages to Person 4. |
| **Meeting cadence** | Short sync (15 min) at the start of each milestone. Use the deliverable list as the agenda. |
| **Handoff protocol** | Before Person 2 can train, Person 1 must confirm features.csv is finalised. Before Person 3 can connect the mod |

## 7. Codebase Structure Reference

| Path | Owner | Purpose |
|---|---|---|
| `data/raw/` | Person 1 | Original downloaded CSVs. Git-ignored. |
| `data/processed/clean_data.csv` | Person 1 | Merged, cleaned dataset. |
| `data/processed/features.csv` | Person 1 | Final feature matrix used for training. |
| `data/processed/airport_resilience_inputs.csv` | Person 1 | Per-airport aggregated stats. |
| `data/processed/resilience_scores.csv` | Person 2 | Computed resilience index per airport. |
| `data/models/disruption_model.pkl` | Person 2 | Trained XGBoost disruption model. |
| `data/models/severity_model.pkl` | Person 2 | Trained multiclass severity model. |
| `notebooks/01_eda.ipynb` | Person 1 | Exploratory data analysis. |
| `notebooks/02_baseline_model.ipynb` | Person 2 | Baseline logistic regression / decision tree. |
| `src/data_prep.py` | Person 1 | Raw data loading, cleaning, merging. |
| `src/features.py` | Person 1 | Feature engineering function. |
| `src/resilience.py` | Person 1+2 | Resilience input aggregation + index computation. |
| `src/model_train.py` | Person 2 | Model training and saving. |
| `src/predict.py` | Person 2 | Prediction functions (imported by dashboard). |
| `src/explain.py` | Person 2 | SHAP explainability + scenario risk functions. |
| `src/recommend.py` | Person 3 | Rule-based recommendation engine. |

| Path | Owner | Purpose |
|---|---|---|
| `app/streamlit_app.py` | Person 3 | Main dashboard application. |
| `assets/shap_summary.png` | Person 2 | Global SHAP beeswarm plot for slides. |
| `tests/test_predict.py` | Person 4 | Smoke tests for prediction functions. |
| `docs/data_sources.md` | Person 1+4 | Dataset provenance and metadata. |
| `docs/feature_dictionary.md` | Person 1+4 | Column definitions — team-wide reference. |
| `docs/kpi_definitions.md` | Person 1+4 | Dashboard KPI formulas and sources. |
| `docs/model_report.md` | Person 2+4 | Model metrics, SHAP results, methodology. |
| `docs/demo_script.md` | Person 3+4 | Step-by-step dashboard demo sequence. |
| `docs/final_report.md` | Person 4 | Full project report for submission. |
| `requirements.txt` | Person 4 | All package dependencies with versions. |
| `README.md` | Person 4 | Project overview and setup instructions. |

---

*This document is an internal team reference for the CLEARS project. All task assignments, file paths, and conventions described here are final as of the project kick-off. Any deviation must be agreed upon by all team members and documented in the relevant docs/ file.*